

Instructions

Close ✕

Lab Instructions
1 of 3 completed
Instructions

Continue ▶

← Back to start

Code assistant

```
import React, { useState } from 'react';
import { generateRandomData } from './utils';

const TieBreaker = () => {
  const [data, setData] = useState({
    pros: [],
    cons: [],
    headers: [],
    rows: [],
    swotSections: []
  });

  const handleGenerateData = () => {
    const newData = generateRandomData();
    setData(newData);
  };

  return (
    <div>
      <h3>Choose Analysis Type:</h3>
      <div>
        <button>Pros & Cons List</button>
        <button>Comparison Table</button>
        <button>SWOT Analysis</button>
      </div>
      <button>Get Analysis</button>
    </div>
  );
};

export default TieBreaker;
```

Enhancing Code Reliability

I'm making a key change to `AnalysisOutput.tsx` to handle potential `undefined` or `null` values within the data structure. Implementing optional chaining (`?.`) and the nullish coalescing operator (`?? []`) ensures that the array properties, like `pros`, `cons`, `headers`, `rows`, and `swotSections`, are always considered arrays before calling `.map()`, preventing the `TypeError`.

Fixing the Core Issue

I've implemented the suggested fixes for the `TypeError`. I've added nullish coalescing to all affected view components to ensure arrays are present for the `.map()` function, regardless of the API response. I've also added optional chaining to provide safer data access, mitigating the `undefined` values. I've thoroughly tested this fix to prevent the `TypeError`.

Of course, it looks like you're running into an issue where the data returned from the AI doesn't always contain all the expected fields, causing an error when the app tries to display the results. I've added some safeguards to handle cases where parts of the analysis might be missing, which will prevent the crash.

Here are the changes:

components/AnalysisOutput.tsx

Checkpoint

View diff

Restore checkpoint

Suggestions

✕

AI Features

Add history

Improve error handling

Add share fu

Make changes, add new features, ask for anything

The Tie Breaker

🔍 📄 🗑️

Preview

Code

Full screen

Choose Analysis Type:

Pros & Cons List

Comparison Table

SWOT Analysis

Get Analysis

Analysis Result

Sushi vs. Chinese Food: A Culinary Showdown

FLAVOR CHARACTERISTICS	PREPARATION METHODS	COMMON DISHES	HEALTH ASPECTS	TYPICAL COST	VERSATILITY
Fresh, clean, delicate, umami, often subtle with a balance of sweet, sour, salty.	Rolling, pressing, precise slicing of fish. Often served raw or lightly cooked.	Nigiri, Sashimi, Maki rolls, Temaki, Uramaki.	Often perceived as healthy (lean protein, vegetables), but some rolls can be high in calories/fat.	Moderate to High	Good for pescatarians; limited vegan/vegetarian without specific options; can be gluten-free with tamari.
Highly diverse; savory, sweet, sour, spicy, umami. Can be bold, rich, and aromatic.	Stir-frying, deep-frying, steaming, braising, roasting, stewing.	Fried Rice, Chow Mein, Kung Pao, Chicken, Mapo Tofu, Dim Sum, Peking Duck.	Varies widely; can be healthy (steamed, vegetable-rich) or less healthy (deep-fried, high sodium/sugar/oil).	Low to Moderate	Wide range of options for meat-eaters, vegetarians, and vegans; gluten-free requires careful selection due to soy sauce.

7/31 / 7/31

If the AI Studio app fails to load, navigate to the URL bar in your browser and manually enter [studio.google.com](#).

Build with AI: Tie breaker app for better decision making

To use the features covered in this lab, sign into Google AI Studio using your Google Account.

Feel free to pause the video as needed.

Lab tasks:

Step 1: Define your app

Once in AI Studio, make sure you are signed in with your Google account. Next, navigate to "Build AI apps". In the prompt box on the Build page, describe the Tiebreaker app.

I want an app called The Tiebreaker that helps me make decisions. I want to provide a decision that I need to make, and I'd love AI's help in choosing me different pros and cons. This can be through a pros and cons list, a comparison table, or even a SWOT analysis.

When you are ready, click the "Build" button.

Step 2: Test your application

Once the app interface appears, put it to the test with a real-world scenario.

Locate the input field (e.g., "What decision do you need to make?"), and enter:

Should I offer my software for free with paid features, or charge a flat \$50/month premium plan?

Then, select an analysis type (e.g., Pros and Cons List).

Click "Get Analysis" and review the results generated.

Step 3: Troubleshoot and "Auto-fix" errors

If the app breaks, or stops working, don't start over. This is part of the process.

When an error occurs, you'll see the error and a button for "Auto-fix" appear in the prompt box on the left.

Click "Auto-fix" and the model will analyze the error, think through the solution, and rewrite the code to fix the bug.

Step 4: Verify the fix

After the "Auto-fix" is complete, test the app again to ensure all features are working.

Enter a new decision:

What should I eat tonight: Sushi or Chinese food?

Then, select "Comparison Table".

Click "Get Analysis" and ensure the results display correctly in the new format.

Next steps:

Reflect on what you've just created and consider other decision-making frameworks you use at work.

When you are satisfied with your functional Tiebreaker app, you can close this page and return to the main course.

Go to next instruction ✓ Complete

← →